SonicBoom: 第三次アウトオブオーダプロセッサ

Jerry Zhao, Ben Korpan, Abraham Gonzalez, Krste Asanovic

英語論文購読第２回　ヴハイナム

Table of Content

1. Introduction

- Deployment of high-performance superscalar, out-of-order (OOO) cores expanding into mobile and edge devices
- Security, power, performance, area need to be considered when evaluating new design
- An open-source hardware implementation of a superscalar, OOO core is invaluable

- Open source hardware implementation have numerous advantages over software models of high-performance cores
  - Demonstrate precise microarchitectural behaviors
  - Execute real applications for trillions of cycles
  - *Empirically* provide power and area measurements
  - Point of comparison for new microarchitectural platform

- Most of the current open-source hardware development framework only support simple in-order cores
  - Rocket
  - Ariane
  - Black Parrot
  - PicoRV32
- Modern mobile / server-class SOC is impossible without a full-featured, high performance implementation of a superscalar OOO core.

***Fastest* open-source** core

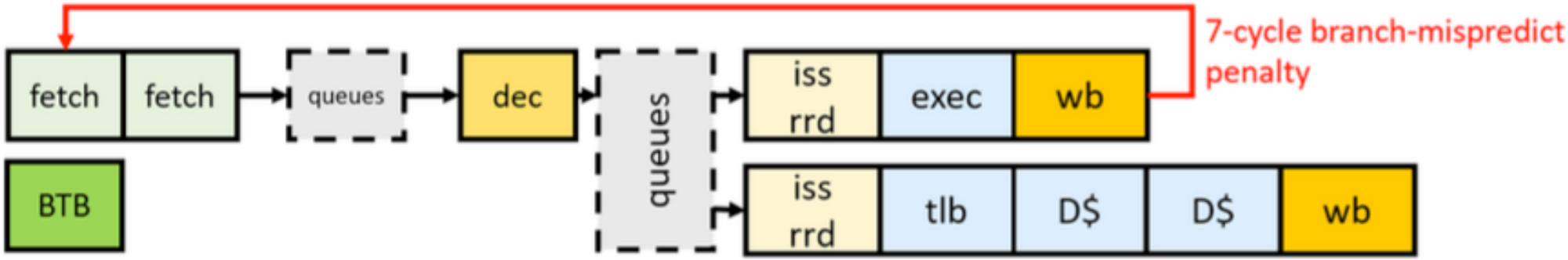
Table 1: Comparison of open-source out-of-order cores.

	***Sonic* BOOM**	**BOOMv2**	**riscy-OOO**	**RSD**
ISA	RV64GC	RV64G	RV64G	RV32IM
DMIPS /MHz	3.93	1.91	?	2.04
SPEC06 IPC	0.86	0.42	0.48	N/A
Branch Predictor	TAGE	GShare	Tourney	GShare
Dec Width	1-5	1-4	2	2
Mem Width	16b/cycle	8b/cycle	8b/cycle	4b/cycle
HDL	Chisel3	Chisel2	BSV+CMD	System Verilog

2. BOOM history

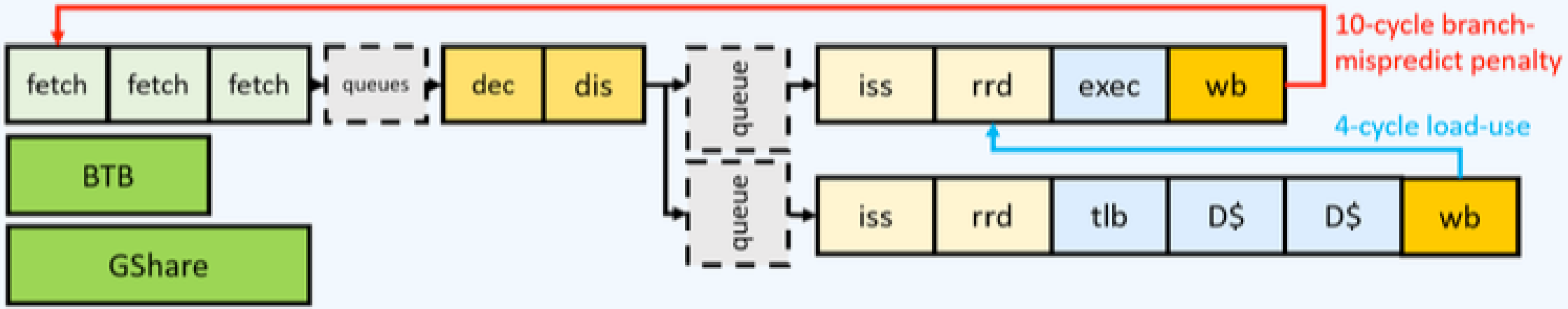
- BOOM Version 1
  - Education tool for University
  - Based on microprocessor MIPS R10K
    (A RISC implementation of MIPS IV ISA)
  - Not enough pipeline stages, thus
    not physically realisable

BOOMv1

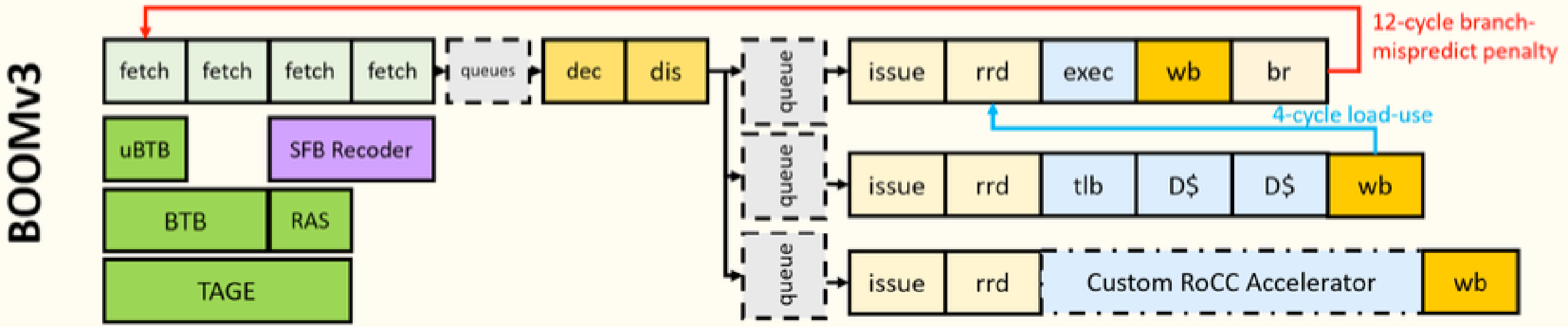

fetch
fetch
queues
dec
queues
iss
rrd
exec
wb
7-cycle branch-mispredict
penalty
BTB
iss
rrd
tlb
D$
D$
wb

- BOOM Version 2
  - Add more pipeline to Front-end and Execution paths
  - Split floating-point register file and execution units to independent pipeline
  - Split issue queues to separate queues: Int, Memory, Float operations

BOOMv2


fetch
fetch
fetch
queues
dec
dis
queue
queue
iss
rrd
exec
wb
10-cycle branch-mispredict penalty
4-cycle load-use
BTB
GShare
iss
rrd
tlb
D$
D$
wb

- Sonic BOOM
  - Support more software stacks
  - Fix performance bottlenecks, keep physical realizability
    - IF
    - EX backend
    - Load / Store
  - Improved critical path delay


BOOMv3
fetch
fetch
fetch
fetch
queues
dec
dis
enq
enq
enq
issue
rrd
exec
wb
br
12-cycle branch-mispredict penalty
4-cycle load-use
issue
rrd
tlb
D$
D$
wb
issue
rrd
Custom RoCC Accelerator
wb
uBTB
SFB Recoder
BTB
RAS
TAGE

- In short
  - If BOOMv1 was created for educational purpose BOOMv3 (SonicBOOM) was created for research in hi-performance core design

3. Instruction Fetch

- Support for 2-byte RVC instructions
- Implementation of TAGE branch predictor
- Multiple branch-resolution units

- Support for 2-byte forms of common 4-byte instructions
  - Default RV-ISA subset for packaged Linux distributions (Fedora, Debian)
  - New fetch-unit which decodes possible 2 / 4 bytes instruction for every 2 bytes

- Implemented branch predictors
  - Get banked to match to ICache
  - Using TAGE and RAS to improve the predictor accuracy, reduce miss aftern returning from a function
- Using multi-branch resolution unit to evaluate multiple B-Instruction per cycle

4. Execute

- Support for RoCC (Rocket Custom Coprocessor) helps integration of custom processor to BOOM pipeline
- Optimized the implementaton of SFB (Short Forward Branch) by recoding difficult-to-predict branches to predicated microOps

- Short Forward Branch:
  - Data-dependent branch over short basic block is common. This lead to high chances of mispredictions and pipeline flushes
  - Short branch get converted to "set-flag" and "conditional-execute" micro-ops
    (e.g. set.ge micro-op)

C	Assembly	Executed MicroOps

`int max = 0;`
`int maxid = -1;`
`for (i = 0; i < n; i++) {`
`  if (x[i] >= max) {`
`  max = x[i];`
`  maxid = i;`
`  }`
`}`
	`loop:`
	`  lw  x2, 0(a0)`
	`  bge  x1, x2, skip`
	`  mv  x1, x2`
	`  mv  a1, t0`
	`skip:`
	`  addi  a0, a0, 4`
	`  addi  t0, t0, 0x1`
	`  j  loop:`
		`loop:`
		`  lw  x2, 0(a0)`
		`  set.ge  x1, x2`
		`  p.mv  x1, x2`
		`  p.mv  a1, t0`
		
		`  addi  a0, a0, 4`
		`  addi  t0, t0, 0x1`
		`  j  loop:`

Branching can be omitted with the use of micro-op

This led to **1.7 times more** Instruction per Cycle

5. Load-Store Unit, Data Cache

L1 Cache drawbacks:

- 1-width interface is a bottleneck of BOOM's fetch and decode pipeline
- Non-speculative caching system
- Blocking load refills on cache eviction
  This leads to increased cycles in evicting the replaced line

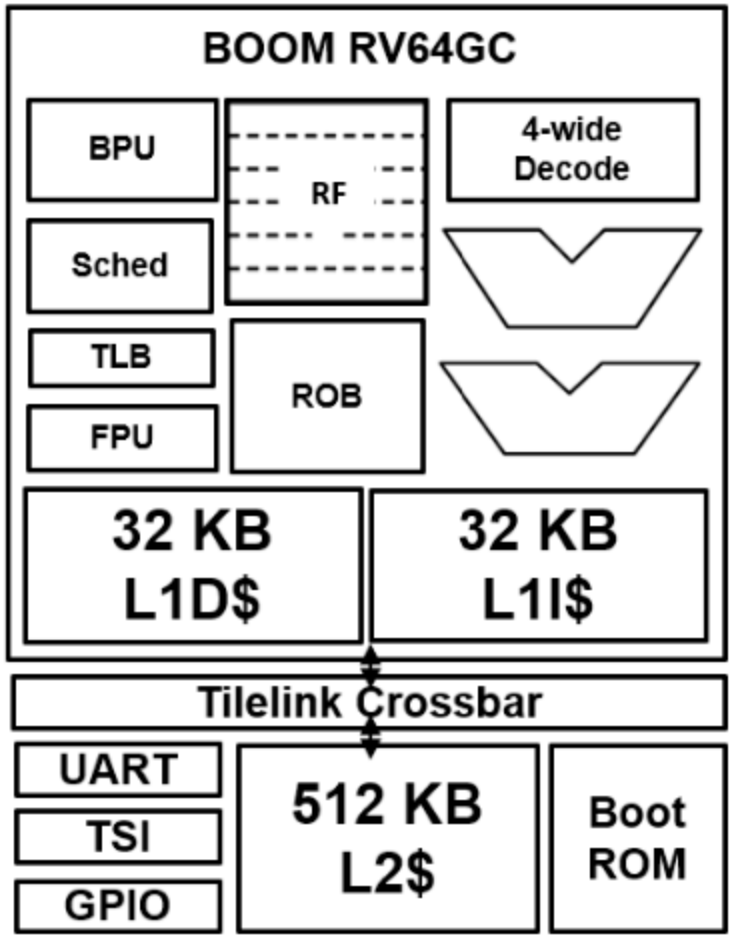
- Dual-ported L1 Data Cache
  - Data cache seperated into 2 banks
    Each bank is a 1R1W SRAM
  - Load-store unit is upgraded accordingly
    to support dual-ported Data cache

- Improved L1 Peformance
  - Cache-eviction can be done in parallel with cache refill. Refill data is written into line-fill buffer
    When Cache-eviction completed, line-fill buffer get flushed into cache arrays
  - Next-line prefetcher between L1 and L2
    Speculatively fetches sequential cache lines after MISS into line buffers
  - Mispeculated cache refills can be flushed out of L1.
    Only write to L1 cache if the request is correctly speculated

6. System Support

- Integrates with many open-source components including UARTs, GPIOs, JTAGs, shared-cache memory system and various accelerators
- A complete BOOM-based SoC can be generated using Chipyard framework
- SonicBOOM high-speed spetaculation helps identified a critical data-race bugs in RISC-V kernel

```
class BaselineConfig extends Config(
  new chipyard.iobinders.WithUARTAdapter     ++
  new chipyard.iobinders.WithBlackBoxSimMem  ++
  new chipyard.iobinders.WithSimSerial       ++
  new testchipip.WithTSI                     ++
  new chipyard.config.WithBootROM            ++
  new chipyard.config.WithUART               ++
  new boom.common.WithMegaBooms              ++
  new boom.common.WithNBoomCores(1)          ++
  new freechips.rocketchip.system.BaseConfig)
```



SonicBOOM based SoC can be created with high-level specification

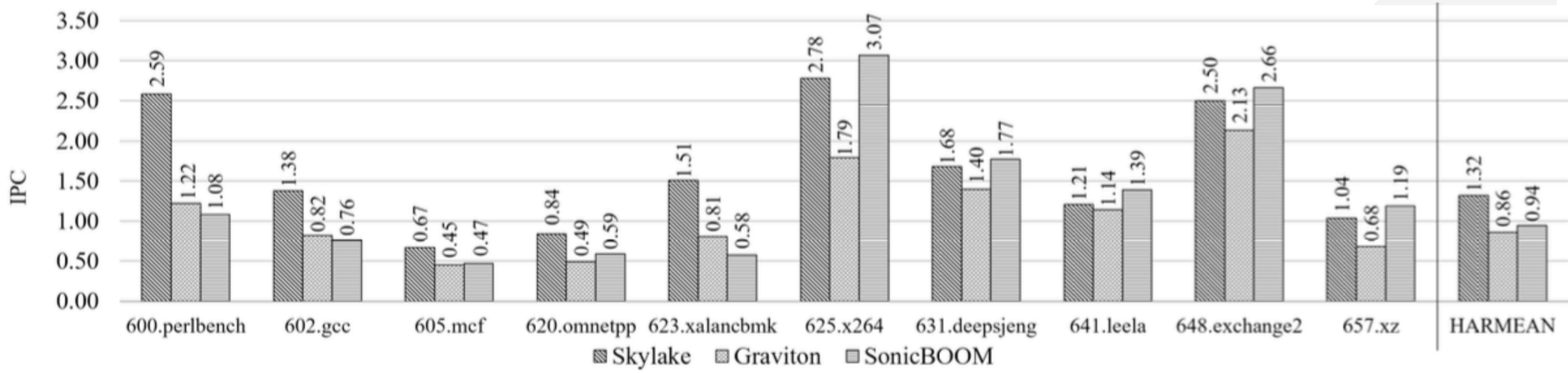
- As an OOO core is very complicated so debugging it is a challenging task.
  Some methods were used to debug it
  - Unit-testing: TraceGen and memtrace were used to test load-store unit and data-cache
  - Dromajo co-simulation tool + FireSim: enable co-simulation at over 1 MHz - magnitudes faster than a software-only co-simulation system

7. Evaluation

Used test suite

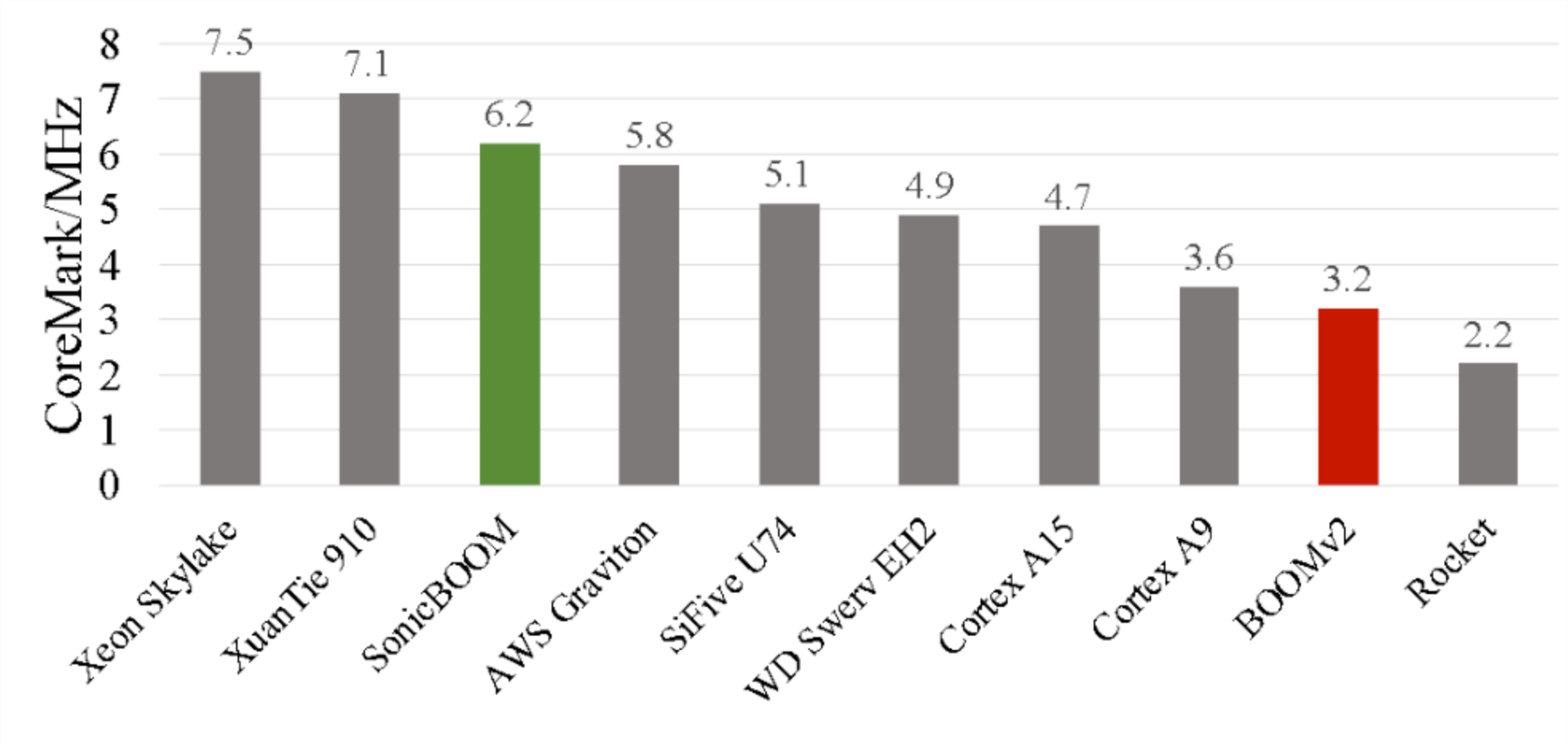
- CoreMark: CPU basic tasks, including string operations, matrix calculation, hash function, state machines. CPU's speed test
- SPECint 2006 CPU: Big integer calculation power. Real-application systematic stress test.

- SPECint test
  - AWS Graviton: Datacenter ARM based 3-wide core
  - Skylake: x86, 6-wide core
- Synthesized SonicBOOM processor is compatitive with Graviton. Can match Skylake on some benchmarks
  - 625.x264: video compression (video stream to H.264/MPEG-4 AVC format)
  - 631.deepsjeng: artificail intelligence benchmark (tree search & pattern recoginition)
- **ISA differences** between 3 systems affected IPC



SonicBOOM is comparable to Graviton (ARM-based) and matches Skylake (x86-based) on some benchmarks

- CoreMark test (Not a good evaluation for out-of-order performance)



SonicBOOM is faster than many prior open-source core

8. What's next

1. Out-of-order implementation of Vector ISA in BOOM
2. L1 miss penalties reduction
    - Implementation of outer-memory prefetcher (instructions & data)

9. Conclusion

- Many performance bottlenecks from BOOMv2 were resolved
- New microarchitectural optimizations were made
- Implemented core is performance-compatitive to datacenters-class cores